

155. Min Stack Medium

Acceptance: 57.6% | [LeetCode](#)

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the `MinStack` class:

- `MinStack()` initializes the stack object.
- `void push(int val)` pushes the element `val` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Example 1:

```
**Input**
["MinStack","push","push","push","getMin","pop","top",
"getMin"]
[[],[-2],[0],[-3],[],[],[],[[]]]
```

```
**Output**
[null,null,null,null,-3,null,0,-2]
```

```
**Explanation**
MinStack minStack = new MinStack();
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); // return -3
minStack.pop();
minStack.top();    // return 0
minStack.getMin(); // return -2
```

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- Methods `pop`, `top` and `getMin` operations will always be called on **non-empty** stacks.
- At most 3×10^4 calls will be made to `push`, `pop`, `top`, and `getMin`.

Claude Opus Solution

Min Stack Solution

1. Intuition

The core challenge is getting the minimum in **$O(1)$** time. A naive approach would scan the entire stack on each `getMin()` call, costing $O(n)$.

Key Insight: We can't avoid scanning *unless we pre-compute and store* the minimum alongside each element. The crucial observation is:

The minimum of a stack only changes when we push or pop. So if we track "what is the minimum at this point in the stack's history," we can answer `getMin()` instantly.

We trade a little extra space to store the running minimum at each level. Since a stack is LIFO, when we pop an element, the minimum naturally "reverts" to what it was before that element was pushed.

2. Approach (Pseudocode)

I'll use the cleanest variant: **a single stack of tuples** `(value, min_so_far)`.

```
push(val):
    if stack is empty:
        current_min = val
    else:
        current_min = min(val, top's stored min)
    stack.push( (val, current_min) )

pop():
    stack.pop()

top():
    return stack[-1].value

getMin():
    return stack[-1].min_so_far    #  $O(1)$  lookup, always at top
```

3. Python Solution

```
class MinStack:
    def __init__(self):
        # Each entry is a tuple: (value, minimum_at_or_below_this_point)
        # By storing the running minimum WITH each element, getMin() is O(1).
        self.stack = []

    def push(self, val: int) -> None:
        # The new minimum is either the value being pushed,
        # or the previous minimum (whichever is smaller).
        if not self.stack:
            current_min = val          # First element: it IS the minimum
        else:
            current_min = min(val, self.stack[-1][1]) # Compare with prev min
        # Store both the value and the min-so-far together.
        self.stack.append((val, current_min))

    def pop(self) -> None:
        # Removing the top automatically "restores" the previous min,
        # since the new top still carries its own stored minimum.
        self.stack.pop()

    def top(self) -> int:
        # The actual value is the first element of the top tuple.
        return self.stack[-1][0]

    def getMin(self) -> int:
        # The min-so-far is the second element of the top tuple – always O(1).
        return self.stack[-1][1]
```

4. Complexity Analysis

Operation	Time	Space
push	O(1)	O(1) extra per call
pop	O(1)	—
top	O(1)	—
getMin	O(1)	—

- **Time:** All operations are O(1). No scanning or searching — every answer is precomputed and sitting at the top of the stack.
- **Space:** O(n) total, where n is the number of elements. We store 2 integers per element instead of 1 (the value + its associated min).

5. Alternative Approaches

A. Two-Stack Approach (common interview answer) Keep a separate `min_stack` that only tracks minimums:

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val):
        self.stack.append(val)
        # Push the smaller of val and current min onto min_stack
        if not self.min_stack:
            self.min_stack.append(val)
        else:
            self.min_stack.append(min(val, self.min_stack[-1]))

    def pop(self):
        self.stack.pop()
        self.min_stack.pop()      # Keep them in sync

    def top(self):
```

```
        return self.stack[-1]

    def getMin(self):
        return self.min_stack[-1]
```

Same complexity, but separates concerns. Easy to reason about.

B. Flattened Single Stack (the reference solution) Interleaves `value` and `min` as alternating entries in one flat list. Clever but harder to read — `top()` must temporarily pop two elements. The tuple approach achieves the same goal more readably.

C. Space-Optimized Min Stack Only push to `min_stack` when a value is `<=` current min, and only pop when the popped value equals the current min. Saves space when there are few new minimums, but adds edge-case complexity (handling duplicate minimums requires the `<=` comparison).

```
def push(self, val):
    self.stack.append(val)
    if not self.min_stack or val <= self.min_stack[-1]:
        self.min_stack.append(val)

def pop(self):
    if self.stack.pop() == self.min_stack[-1]:
        self.min_stack.pop()
```

Recommendation

For interviews, the **tuple approach** (my main solution) is ideal — it's the clearest demonstration of the "precompute the answer" insight while keeping all operations trivially $O(1)$.

Codex Solution

1. Intuition

A normal stack can already support:

- push in $O(1)$
- pop in $O(1)$
- top in $O(1)$

The difficult operation is:

- getMin in $O(1)$

If we scan the whole stack every time to find the minimum, `getMin` would take $O(n)$, which is not allowed.

Key Insight

When we push a value, we can also remember the minimum value of the stack **at that exact moment**.

So instead of storing only:

```
value
```

we store:

```
(value, current_minimum)
```

For example:

```
push(-2) -> stack: [(-2, -2)]
push(0)  -> stack: [(-2, -2), (0, -2)]
push(-3) -> stack: [(-2, -2), (0, -2), (-3, -3)]
```

Now the minimum of the entire stack is always stored alongside the top element.

Therefore:

```
getMin()
```

just returns the second value of the top pair.

2. Approach

We maintain one stack.

Each stack entry stores two values:

```
[value, minimum_so_far]
```

Pseudocode

```
class MinStack:

    initialize:
        stack = empty list

    push(val):
        if stack is empty:
            current_min = val
        else:
            current_min = min(val, stack top's minimum_so_far)

        push [val, current_min] onto stack

    pop():
        remove top element from stack

    top():
        return stack top's value

    getMin():
        return stack top's minimum_so_far
```

3. Python Solution

```
class MinStack:
    def __init__(self):
        """
        Initialize the stack.

        Each element in self.stack will be a pair:
        [value, minimum_so_far]

        value:
            The actual value pushed onto the stack.

        minimum_so_far:
            The minimum value in the stack at the time this value was pushed.
        """
        self.stack = []

    def push(self, val: int) -> None:
        """
        Push a new value onto the stack.

        We also store the minimum value seen so far.

        Example:
            If the current minimum is -2 and we push -3,
            the new minimum becomes -3.

            If the current minimum is -2 and we push 0,
            the minimum remains -2.
        """

        # If the stack is empty, the new value is also the minimum.
        if not self.stack:
            current_min = val
        else:
            # The previous minimum is stored at the top of the stack.
            previous_min = self.stack[-1][1]
```

```

    # The new minimum is the smaller of:
    # 1. the new value
    # 2. the previous minimum
    current_min = min(val, previous_min)

    # Store both the value and the minimum so far.
    self.stack.append([val, current_min])

def pop(self) -> None:
    """
    Remove the top element from the stack.

    Since each element already stores the minimum at that point,
    removing the top element automatically restores the previous minimum.
    """
    self.stack.pop()

def top(self) -> int:
    """
    Return the top value of the stack.

    The actual value is stored at index 0 of the top pair.
    """
    return self.stack[-1][0]

def getMin(self) -> int:
    """
    Return the minimum value in the stack.

    The current minimum is stored at index 1 of the top pair.
    """
    return self.stack[-1][1]

```

4. Complexity Analysis

Let n be the number of elements in the stack.

Time Complexity

Operation	Complexity	Explanation
push	$O(1)$	We do a constant number of operations
pop	$O(1)$	We remove the last element
top	$O(1)$	We access the last element
getMin	$O(1)$	The current minimum is stored at the top

Overall, every function runs in:

$O(1)$

Space Complexity

$O(n)$

We store one pair for every pushed element.

Each element stores:

[value, minimum_so_far]

So the stack uses linear extra space.

5. Alternative Approaches

Alternative 1: Two Stacks

Use:

```
self.stack
self.min_stack
```

- `self.stack` stores all values.
- `self.min_stack` stores the current minimum after each push.

Example:

```
stack:      [-2, 0, -3]
min_stack:  [-2, -2, -3]
```

Then:

```
getMin()
```

returns:

```
min_stack[-1]
```

This is also $O(1)$ time for all operations and $O(n)$ space.

Alternative 2: Optimized Two Stacks

Instead of pushing the minimum every time, only push to `min_stack` when the new value is less than or equal to the current minimum.

Example:

```
push(x):
    stack.append(x)

    if min_stack is empty or x <= min_stack[-1]:
        min_stack.append(x)
```

When popping, if the popped value equals the current minimum, also pop from `min_stack`.

This can use less space in practice, but worst-case space is still:

```
 $O(n)$ 
```

Alternative 3: One Stack with Encoded Values

There is a trick where you store encoded values and keep a separate variable for the current minimum.

This can achieve $O(1)$ operations and use only one stack plus one variable. However, it is more complex and can have integer overflow concerns in languages with fixed-size integers.

For Python, the pair-based solution is usually the cleanest and safest.